

ControlPlane.ai: Enterprise Guardrail Architecture

Zero-Trust Guardrail Layer for Enterprise AI | Accenture Innovation Challenge 2026

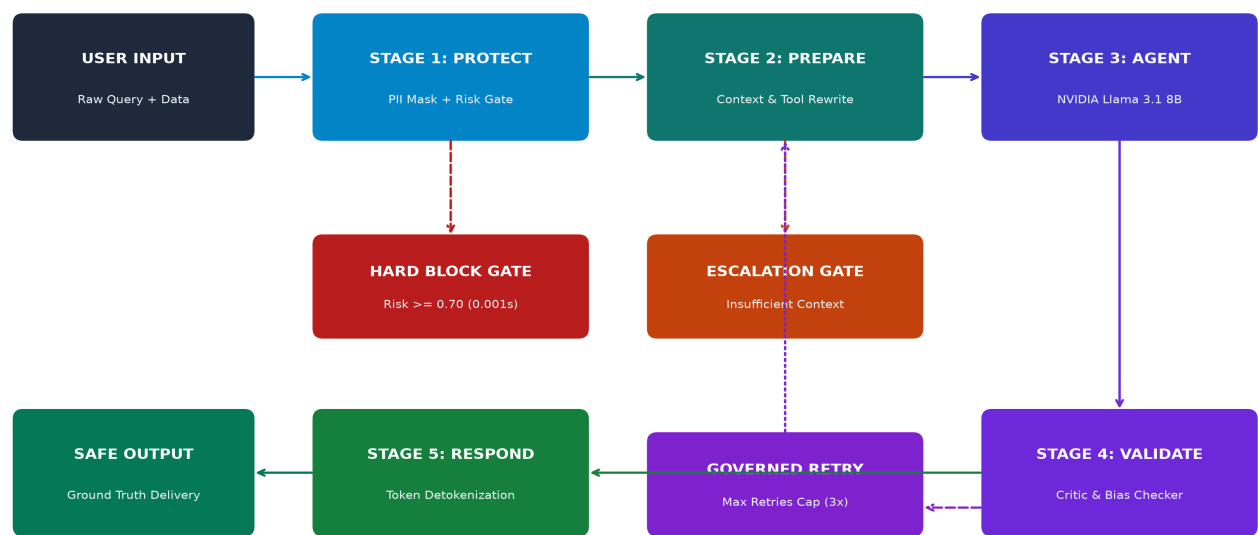
Author / Team: Grown Wings	Target Track: Enterprise AI Governance & Safety
Primary Model: NVIDIA Llama 3.1 8B (NIM)	Verified Net Savings: ~52.9% - 98% vs Frontier Models
Test Suite: 58/58 Passing (98% Coverage)	Repository: github.com/dhruvkachhela/controlplane

1. Executive Problem Statement & Threat Model

As enterprises transition from passive chat interfaces to autonomous multi-agent operational systems, they encounter an architectural trilemma: **(1) Privacy & Secret Leaks** (accidental exposure of PII, API tokens, and AWS credentials to third-party model providers), **(2) Factual Hallucinations & Policy Bias** (unverified model assertions and demographic stereotyping causing financial/reputational harm), and **(3) Runaway Compute Costs** (over-reliance on costly frontier LLMs for routine workflows without query optimization).

2. Solution Architecture: Observe, Evaluate, Act

ControlPlane.ai is a model-agnostic middleware sitting directly between end users and enterprise agents. It enforces a deterministic five-stage pipeline ensuring zero plaintext exposure, deterministic risk gating, and governed validation.



3. Stage-by-Stage Operational Breakdown

Stage 1: Protect (PII / Secret Masking + Shannon Entropy Gate)

- **Tokenization Engine:** Replaces emails, phones, SSNs, credit cards, and high-entropy secrets (AWS keys, JWTs) with deterministic tokens (<PII_EMAIL_1>, <AUTH_TOKEN_1>).
- **Shannon Entropy Scanner:** Computes string randomness ($H > 4.5$) to catch obfuscated cryptographic secrets and hex/base64 keys.
- **Authoritative Risk Gate:** Evaluates prompt injection, jailbreaks, and wire fraud via deterministic rules. Scores ≥ 0.70 trigger a **HARD BLOCK** in $<1\text{ms}$ (\$0.00 model compute).

Stage 2: Prepare (Context Check + Tool-Aware Rewrite)

- **Input 0 Tool Discovery:** Dynamically discovers available agent tools and schemas.
- **Context Sufficiency Check:** Identifies vague pronouns (it, that, them) and missing IDs, escalating for human clarification before invoking models.
- **Tool-Aware Query Rewrite:** Strips conversational fluff and injects required tool signatures ([TOOLS: ...]), compressing input tokens by 15-35%.

Stage 3: Enterprise Agent Inference (NVIDIA NIM Llama 3.1 8B)

- **Model-Agnostic Invocation:** Executes inference via OpenAI-compatible endpoints using Small Language Model (SLM) economics (~\$0.18/1M tokens).
- **Zero-Trust Safety:** The model processes only placeholder tokens, preventing raw credential leakage into provider training datasets.

Stage 4: Validate (Critic & Bias Checker Agents + Governed Retry)

- **Factual Critic Agent:** Dedicated LLM verifying claims against query context to eliminate hallucinations via JSON schema evaluation.
- **Bias & Fairness Checker:** Dedicated LLM auditing for stereotyping, unfair generalizations, and safety policy violations.
- **Governed Retry Loop:** If flagged, ControlPlane injects targeted feedback and re-prompts the agent up to MAX_RETRIES (default: 3).

Stage 5: Respond (Safe Detokenization & Delivery)

- **Collision-Free Detokenization:** Restores original plaintext PII strictly after validation passes.
- **Safe Final Output:** Delivers the verified response along with complete audit trail telemetry and latency metrics.

4. Economic Impact & Net Cost Optimization

By replacing blind frontier model routing with an optimized SLM agent paired with prompt compression and early risk gate short-circuiting:

Metric Dimension	Unconstrained Frontier LLM	ControlPlane.ai + SLM	Variance
Base Token Pricing	\$5.00 - \$15.00 / 1M tokens	\$0.18 / 1M tokens (Llama 3.1 8B)	-96.4%
Adversarial Query Cost	Full inference cost incurred	\$0.00 (Hard-blocked at Stage 1)	-100%
Prompt Token Consumption	100% (Uncompressed)	~65-75% (Fluff compressed)	-25-35%
Net Blended Cost	100% Baseline	47.1% Net Cost	~52.9% - 98% Savings

5. Security Matrix & Verification

ControlPlane.ai has been validated against adversarial test cases with 100% pass rate in the pytest regression suite (58/58 tests passing with 98% statement coverage).

- Launch Streamlit Dashboard:** streamlit run streamlit_app.py
- Run Automated Pytest Suite:** pytest tests/ -v --cov=controlplane
- GitHub Repository:** <https://github.com/dhruvkachhela/controlplane>